A2 学生版讲义 · 服务端 Socket 与登录链路

> **课程**:CS 实训(C/S 模式 Java 图形界面开发)· Vibe Coding 单元② **用法**:课前预习、课后复习都能看。内容精炼,配合课堂消化。 **今天开始施工**:先从零补 **Socket + 线程** 两块基础,再拿单元①的图纸让 AI 把服务端盖出来,打通"登录"这一条 C/S 链路。 **学情提示**:今天用到的 **网络编程 / Socket / TCP** 和 **多线程** 你还没在课上学过——本讲义二、三节从零讲,配最小代码,先看懂再动手。

一、今天的核心:先补基础,再通一条链路

单元①你写好了图纸(`.trae/rules`)。今天施工第一步不是画界面,而是**把 C/S 的命脉通道打通**:客户端发一行文本 → 服务端查库 → 服务端返一行文本。

但图纸里的"每客户端一线程""发一行收一行",用到的是 **Socket(网络编程)**和**多线程**——这两块你还没学过。所以今天前半程**从零补上**(二、三节),你才看得懂 AI 生成的服务端。

为什么先做登录? 因为它是最短、最完整的一条链路。把它跑通,等于把整条 C/S 通道验证了一遍。后面查书、借书、还书,全是**复制这条链路换业务**。

> 🔑 C/S 心法:**先补基础(Socket+线程)→ 跑通最小 echo → 再通一条链路(登录)→ 复制换业务。别先堆界面。**

二、Socket 零基础速记(没学过网络编程也能懂)

两个程序怎么"连上说话"——靠三样东西定位和建立通道:

词	是什么	生活类比
IP 地址	哪一台机器(`127.0.0.1` = 本机自己)	小区**门牌号**
端口	机器上哪个程序(服务端 9000 / MySQL 3306)	门牌里的**房间号**
TCP	"先拨通、再说话"的可靠通信	打**电话**:先接通,再一句来一句去,说完挂断

> 🔑 **IP 找机器,端口找程序,TCP 负责"先拨通再说话"。** `127.0.0.1:9000` = 本机这栋楼的 9000 房间。

Socket 编程的几个词(把 Socket 理解成"一条已接通的电话线"):

词	干什么	类比
`ServerSocket`	服务端在某端口"守着等电话":`new ServerSocket(9000)`	总机
`Socket`	一通接通的电话 / 一条连接,两边各拿一个	话筒
`accept()`	**接起一通电话,返回一个 `Socket`;会阻塞**(停着等)直到有人连上	接电话
输入流 `getInputStream`	读对方的话 → 包 `BufferedReader`,`readLine()` **读一行**	听
输出流 `getOutputStream`	写给对方 → 包 `BufferedWriter`,`write()` + `"\n"` + `flush()` **写一行**	说

> 📌 **心智模型**:一条 socket 连接 = **"一根能 `readLine()` 读一行、能 `write()` 写一行的管子"**。今天所有网络代码都在这根管子上读一行、写一行。中文不乱码 → 读写流都用 `UTF-8`。

三、线程零基础速记(没学过多线程也能懂)

问题:一个程序 `accept()` 到一个客户端后,就在循环里陪它收发,**主线程被占住**,回不到 `accept()` 接下一个——**单线程一次只能服务一个人**。

词	是什么	生活类比
进程	运行 `java LibraryServer` 开的一块独立地盘	一家营业的**饭店**
线程	进程里一条能独立跑的执行流,可有很多个、**同时**干活	饭店里一个**服务员**

怎么起一个线程(两种写法):

```
// 写法一:Runnable(lambda 简写)
Thread t = new Thread(() -> { /* 要干的活 */ });
t.start();   // ⭐ start() 才是"另起一条线程并行跑"

// 写法二:实现 Runnable 接口(我们的 ClientHandler 就是这种)
class ClientHandler implements Runnable {
    public void run() { /* 陪一个客户端收发 */ }
}
new Thread(new ClientHandler(socket)).start();
```

> ⚠️ **最容易错**:`start()` 才是起新线程并行;`run()` 只是普通调一次方法,**不并行**——写成 `run()` 等于没开线程,还是只能服务一个人。审查 AI 时要盯这一点。

四、先跑通一个 echo demo(把上面理论落地)

图书馆服务端 = echo demo + 查数据库 + 每客户端一线程。 先跑通最小的 echo(服务端收到什么原样发回),亲眼看 socket 通信发生。

- 先起 `EchoServer`(打印"等客户端连接"后**停住 =** **`accept()`** **在等**,不是卡死),再起 `EchoClient`。
- 客户端打印 `服务端回复:你说的是:你好,服务端` = 一次完整的"发一句收一句"。
- 再开**第二个** `EchoClient` → **没反应**(单线程只服务一个)= 下一步要用"每客户端一线程"解决的问题。

```
// EchoServer 核心
ServerSocket serverSocket = new ServerSocket(9000); // 总机守 9000
Socket socket = serverSocket.accept();              // 阻塞等,返回一条连接
BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
BufferedWriter out = new BufferedWriter(new
OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8));
String line;
while ((line = in.readLine()) != null) {            // 读一行
    out.write("你说的是:" + line); out.write("\n"); out.flush(); // 原样回一行
}
```

五、C/S 三段式:一次登录,数据怎么流

```
[Swing 客户端]  ──①发一行──▶  [多线程服务端]  ──②JDBC查──▶  [MySQL]
  LOGIN|admin|123456            ClientHandler          SELECT ... FROM t_user
[Swing 客户端]  ◀──④回一行──  [多线程服务端]  ◀──③返回──  [MySQL]
  OK|admin|系统管理员|1          拼 OK|role|realName|userId
```

四步,记死这一句:**客户端发一行字符串 → 服务端查库 → 返一行字符串。**("发一行收一行"就是第四节 echo demo 那根管子,这里只多插了一步"查数据库"。)

- 客户端**不碰数据库**,连库地址都不知道;
- 服务端是**唯一的守门人**,所有请求都过它、地址密码只在它这;
- 客户端拿到那一行响应,再决定打开哪个界面。

六、⭐ 为什么"每客户端一线程"

图书馆同一时刻很多人在用(张三查书、李四借书、王五登录)。服务端只有一个,怎么同时招呼?

每来一个客户端,就派一个线程专门服务它(就是第三节"每桌一个服务员")。

- **`accept()`** **是阻塞的**:主线程停在那里等,有客户端连上来才返回一个 `Socket`。
- **拿到连接立刻开线程**:`new Thread(new ClientHandler(socket)).start()`(是 `start()` 不是 `run()`),把这个客户端交给新线程,主线程马上回到 `accept()` 等下一个。
- **不开线程会怎样?** 主线程陪第一个人陪到底,后面的人**没人接**(就是 echo demo 第二个客户端没反应)——服务端一次只能服务一个人。这是初学者和 AI 都容易踩的坑。

> 💬 比喻:服务端门口有个迎宾(主线程),`accept()` 就是他在门口等客人。客人一到,他不自己陪到底,而是**叫一个服务员(新线程)专门带这桌**,自己转身回门口迎下一位。这样一百桌同时来也不乱。

七、自定义文本协议:两边怎么"说话"

隔着网络,得先约定用什么格式说话——这就是协议。我们自己定一套极简文本协议:

- **请求**:一行一条,命令开头,参数用 `|` 分隔——`LOGIN|admin|123456`、`BOOK_QUERY|Java|1`、`BORROW|2|5`。
- **响应**:也是一行——成功 `OK|数据`,失败 `ERR|原因`。
  - 登录成功:`OK|admin|系统管理员|1`(角色 | 真实姓名 | 用户 id)
  - 登录失败:`ERR|账号或密码错误`
- **列表数据要转义**:书名里若有 `|` 会把协议拆乱,所以图书列表这类数据由手写的极简 `Json` 工具编码成"不含裸 `|`、不含换行"的一行再传(今天登录还用不到,查书时才用)。

> 协议是 C/S 的"普通话"。`ClientHandler` 就是在那根管子读到的一行文本上,按 `|` 拆开看是什么命令。

八、服务端查库:PreparedStatement 防注入

服务端拿到 `LOGIN|admin|123456`,用 **JDBC** 去查 `t_user`。关键一条底线:

- **用 `PreparedStatement`(`?` 占位),不要拼字符串**:`SELECT ... FROM t_user WHERE username = ? AND password = ?`,再 `ps.setString(1, username)` 填值。
- **为什么?防 SQL 注入**:拼字符串的话,有人把密码输成 `' OR '1'='1`,SQL 就变成"永远成立",不用密码也能登进去。用 `?` 占位,数据库把它当**纯数据**处理,注入就失效了。

> 这是审查 AI 的一个重点:它写查库 SQL 有没有老实用 `?` 占位。

九、生成 prompt 速查(可复制)

Step 0 · 先跑通 echo demo

```
用纯 Java 11 写一个最小的 Socket echo(回声)demo,不用任何框架,分成两个可直接运行的类:
- EchoServer:用 ServerSocket 监听 9000;accept() 等一个客户端;从 socket 拿输入流(BufferedReader)
  和输出流(BufferedWriter),流都用 UTF-8;循环 readLine() 收一行,原样加前缀"你说的是:"写回,记得 write 换行 + flush。
- EchoClient:用 Socket 连 127.0.0.1:9000,发一行"你好,服务端",再 readLine() 读回复并打印。
只要这两个类,能直接编译运行。
```

Step 1 · 建库 + 种子

```
按 .trae/rules 里的数据模型,给我一段 MySQL 建库脚本:库名 library_cs(DEFAULT CHARSET utf8mb4),
建 t_user、t_category、t_book、t_borrow 四张表(字段照 rules,带主外键),插入种子数据:3 个分类、6-8 本图书、管理员 admin/123456、读者 zhangsan/123456 和 lisi/123456。只输出 SQL,我去 MySQL 执行。
```

Step 2 · 服务端骨架(Socket + 多线程 + DBUtil)

```
帮我创建 Java 服务端(纯 Java 11,不用框架):
- DBUtil:JDBC 连 127.0.0.1:3306/library_cs(root/12345678),url 带 characterEncoding=UTF-8(不要写 utf8mb4)、useSSL=false、serverTimezone=Asia/Shanghai;提供 getConnection()。
- LibraryServer:ServerSocket 监听 9000,accept 到一个客户端就 new Thread(new ClientHandler(socket)).start()——每客户端一个线程。
- ClientHandler:循环 readLine() 收一行请求,先原样回显(先不接业务)。
- 一个手写的 Json/编码工具类,把 List<Map> 编成一行文本(含 | 和换行要转义),不引第三方库。
先只搭骨架:能启动监听、能收到客户端一行并回显。分层放 server/、server/util/、common/util/。
```

Step 3 · 打通登录链路

```
基于现有服务端,实现"登录"这一条完整链路:
- 定义协议:客户端发 LOGIN|用户名|密码;
- ClientHandler 解析命令,登录走 UserService.login → UserDao 用 PreparedStatement 查 t_user;
- 成功返回 OK|role|realName|userId,失败返回 ERR|账号或密码错误。
加 server/service/UserService、server/dao/UserDao、common/entity/User。只做登录。
```

Step 4 · 顺手闭合注册链路(REGISTER)

> 登录通了,但读者账号都是种子数据手插的。真实图书馆读者得能**自己注册**。注册和登录是**同一条通道换业务**:`REGISTER|用户名|密码|姓名` → 服务端**查重 + 插库** → 返 `OK|新用户id`。承接 Step 3 同一对话继续追加:

```
基于现有服务端的登录链路,再补一条"读者注册"完整链路,协议命令用 REGISTER:
- 协议:客户端发 REGISTER|用户名|密码|真实姓名,成功返回 OK|新用户id,失败返回 ERR|原因;在 common/Protocol 里加 REGISTER 常量。
- 服务端:ClientHandler 加 REGISTER 分支 → UserService.register:先校验用户名和密码非空,再用 UserDao.existsUsername 查重(用户名已存在就抛异常提示"用户名已存在"),通过后 UserDao.insertReader 插入,role 固定 'reader'、max_borrow 默认 5(注册的一律是读者,不能是 admin),返回自增主键。所有 SQL 都用 PreparedStatement。
- 客户端:一个 Swing 注册对话框 RegisterDialog(用户名/密码/确认密码/真实姓名四个输入框),提交前在界面上校验"用户名密码非空""两次密码一致",再调 Api.register 发 REGISTER;成功弹"注册成功,请用新账号登录"。
t_user 的 username 已是 UNIQUE 唯一索引(Step 1 建库脚本里有),注册重名要走 existsUsername 提前拦。
```

注册链路审查要点(登录没有、注册新加的把关功夫):

审查点	为什么
写库前先查重	顺序是"先非空校验 → 再 `existsUsername` 查重 → 最后 `insertReader` 插入",不能先插再查重。AI 若漏查重直接插库,撞 `username` UNIQUE 索引会返回一长串 `Duplicate entry` 数据库异常栈,用户看不懂
两层校验都要	界面校验(非空 / 两次密码一致)是**体验**,服务端校验(非空 / 查重)是**安全底线**。懂行的人绕过界面直接发协议,服务端那道才拦得住——**只靠界面 = 服务端裸奔**
`role` 写死 `'reader'`	INSERT 里 role 固定 `'reader'`、不接客户端传参,否则有人能注册成 admin
`username` UNIQUE 兜底	唯一索引是并发下最后一道防线——两个请求可能同时通过 Service 查重双双插入,查重不是原子的

> ⚠️ **迭代练习(强制至少 1 轮)**:注册跑通后,同一对话追加"加学号(UNIQUE、必填查重)+ 邮箱格式校验"。AI 大概率把邮箱正则**只放界面**——追问"服务端 register 是否也校验了?界面能被绕过",逼它把校验补到 Service 层。

端到端闭合验证:

1. 发 `REGISTER|wangwu|abc123|王五` → 收到 `OK|4`(新自增 id);
2. `SELECT * FROM t_user WHERE username='wangwu'` → `role='reader'`、`max_borrow=5`;
3. **闭合**:紧接着发 `LOGIN|wangwu|abc123` → 收到 `OK|reader|王五|4`(刚注册的账号能登录了);
4. 再发一次 `REGISTER|wangwu|xxx|王五2`(重名)→ 收到 `ERR|用户名已存在`(不是异常栈);
5. 发 `REGISTER|zhaoliu||赵六`(密码空)→ 收到 `ERR|用户名和密码不能为空`。

> 💬 注册没让你学任何新的 Socket / 线程知识——它就是把登录那条通道**复制一遍换业务**。你真正新练的是"写库前先查重""校验要两层""唯一索引兜底"这几条**审查和把关**的功夫,这才是 Agent 时代你比 AI 值钱的地方。

十、四层职责表(看懂生成的代码)

层	类	干什么	一句话
连接	`DBUtil`	给一个数据库连接	地址密码只在这(`characterEncoding=UTF-8`)
监听	`LibraryServer`	监听 9000 + 派线程	`accept()` 阻塞等人,拿到连接就 `start()` 开线程
收发	`ClientHandler`	收一行、按命令分发、回一行	一个客户端一个它(一个线程)
查库	`UserDao`	用 `PreparedStatement` 查 `t_user`	`?` 占位防注入,查到返 `User`

(业务层 `UserService` 夹在 `ClientHandler` 和 `UserDao` 之间,做登录校验;实体 `User` 对应 `t_user`。)

十一、两个高频坑(现场必踩)

坑	现象	修法
`characterEncoding` 写成 utf8mb4	一连库报 `Unsupported character encoding`	改成 `characterEncoding=UTF-8`(Java 字符集名);数据库层 utf8mb4 不动
忘了 `new Thread(...).start()` 或写成 `run()`	第二个客户端连上来没反应,只能服务一个人	`accept()` 后用 `start()` 开线程处理,主线程回到 `accept()` 等下一个

> 注意:**建库脚本**用 `utf8mb4`(MySQL 字符集名),**JDBC 连接串**用 `characterEncoding=UTF-8`(Java 字符集名),两个位置别搞混。

十二、验证登录链路(今天的验收)

让 AI 写个 5 行的测试客户端(其实就是把 echo demo 的 `EchoClient` 改一句要发的话),连 `127.0.0.1:9000`,发 `LOGIN|admin|123456`,打印返回。三条都做到算通:

1. 服务端启动打印"监听端口 9000";
2. 发 `LOGIN|admin|123456` 收到 `OK|admin|系统管理员|1`;
3. 发错误账号密码收到 `ERR|账号或密码错误`。

十三、FAQ

Q1:IP、端口、TCP 到底各管什么? IP 找机器(`127.0.0.1` 就是本机自己),端口找机器上的哪个程序(我们服务端用 9000),TCP 负责"先拨通连接再一句来一句去地说话"。合起来:客户端"拨通本机的 9000 房间"。

Q2:`ServerSocket` 和 `Socket` 有什么区别? `ServerSocket` 是服务端的"总机",守在端口等电话(`new ServerSocket(9000)`);`Socket` 是一通"接通的电话",代表一条连接。`accept()` 接起电话、返回一个 `Socket`,之后就用这个 `Socket` 上的输入/输出流读一行、写一行。

Q3:`accept()` 为什么会"卡住"? `accept()` 是阻塞方法:没客户端连上来时它就停在那里等,直到有人连上才返回一个 `Socket`。这是正常行为,不是卡死——去把客户端跑起来它就往下走了。

Q4:起线程为什么必须用 `start()` 不能用 `run()`? `start()` 才是"另起一条新线程并行跑",主线程不等它继续走;`run()` 只是在当前线程里普通调一次方法,不并行。写成 `run()` 就等于没开线程,服务端还是只能服务一个人。

Q5:为什么每来一个客户端都要开线程? 不开的话,主线程陪第一个客户端陪到底,后面的人没人接,服务端一次只能服务一个人(echo demo 里你能亲眼看到)。开了线程,多个读者天然并行、互不阻塞。这是答辩最爱问的点。

Q6:`characterEncoding` 到底填什么? 填 `UTF-8`(Java 的字符集名)。`utf8mb4` 是 MySQL 的字符集名,填进 JDBC 连接串 Java 不认,会报 `Unsupported character encoding`。数据库/建表用 utf8mb4 是对的,别混。

Q7:登录用 `PreparedStatement` 有什么讲究? `?` 占位 + `setString` 填值,数据库把输入当纯数据处理,防 SQL 注入。绝不能把用户名密码拼进 SQL 字符串,否则有人输 `' OR '1'='1` 就能绕过密码。

Q8:今天 `Json` 工具用不上,为什么还要生成? 它是为后面查书、借阅记录这类**列表数据**准备的——把多行数据编码成不含裸 `|`/换行的一行,避免拆乱协议。今天登录返回是简单的 `OK|role|realName|userId`,还用不到,先生成放着。

今日产出

从零补上 **Socket**(IP/端口/TCP、`ServerSocket`/`Socket`/`accept`/流)和 **线程**(进程vs线程、`Thread`/`Runnable`/`start`)两块基础;跑通 **echo demo** 亲眼看 socket 通信;讲清 **C/S 三段式通信 + 为什么每客户端一线程 + 文本协议 + PreparedStatement 防注入**;用 SOLO 分步生成**建库 + Socket 多线程服务端骨架 + 登录链路**,并**顺手闭合 REGISTER 注册链路**(同一通道换业务:查重 + 插库,端到端"注册→用新账号登录→收 OK");看懂四层职责、审查四条关键点 + 注册新增的"写库前查重 / 双层校验"、验证 `LOGIN|admin|123456` 收到 `OK|admin|系统管理员|1` ——**C/S 那条命脉通道打通了**,后面四个单元都往这根骨架上加肉。